# AGENTS.md

## 1.什麼是 AGENTS.md？

**AGENTS.md 是放在軟體專案中的 Markdown 指令檔，用來告訴 Agent「這個專案要怎麼工作」。** 你可以把它理解成「給 AI Agent 看的 README」：README.md 主要服務人類開發者，而 AGENTS.md 則放入 AI 在修改程式時需要持續遵守的專案規則，例如建置方式、測試指令、程式風格、可修改範圍、安全限制與完成條件。

> Prompt 是「這次要做什麼」；AGENTS.md 是「在這個專案裡，一直要怎麼做」。

### 官方文件

**OpenAI Codex：Custom instructions with AGENTS.md**
https://developers.openai.com/codex/guides/agents-md

**AGENTS.md 開放格式網站**
https://agents.md/

**OpenAI：Codex Prompting Guide**
https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide

## 2. 為什麼需要 AGENTS.md？

使用 Agent 時，我們常會重複提醒相同規則：不要改資料檔、先跑測試、使用特定套件、不要部署到正式環境、UI 風格不要改動等。若每次都重新輸入，不但浪費時間，也容易漏掉。AGENTS.md 將這些「專案長期規則」留在程式碼旁，讓 Agent 進入專案時就取得一致的工作背景。

| 項目 | 主要讀者 | 適合放什麼 | 生命週期 |
|---|---|---|---|
| README.md | 人類開發者 | 專案介紹、安裝、使用方式 | 專案長期文件 |
| AGENTS.md | Agent | 開發規範、測試、限制、完成條件 | 專案長期指令 |
| Prompt / 任務訊息 | Agent | 這一次要完成的工作 | 單次任務 |

### AGENTS.md 常見內容

- Project overview：專案用途、主要技術與重要目錄。
- Setup / Build：安裝依賴、啟動、建置指令。
- Testing：修改後要跑哪些測試、什麼才算通過。
- Code style：語言版本、命名、格式與架構慣例。
- Boundaries：哪些檔案不能動、哪些操作需要先詢問。
- Definition of done：完成前要檢查什麼、輸出什麼。

> AGENTS.md 沒有強制欄位；它就是一般 Markdown。

## 3. Codex 會怎麼讀取 AGENTS.md？

Codex 在開始工作前會建立一條 instructions chain。它可以同時讀取「全域規則」與「專案規則」，而越接近目前工作目錄的規則優先度越高。

**讀取概念**

```
~/.codex/AGENTS.md        ← 個人全域規則
my-project/AGENTS.md      ← 整個專案規則
my-project/frontend/AGENTS.md ← frontend 子專案規則
my-project/frontend/src/  ← 若從這裡工作，會合併上面規則
```

- 全域：Codex home（預設 ~/.codex）可以放 AGENTS.md；若同層有 AGENTS.override.md，override 優先。
- 專案：Codex 從專案根目錄一路往目前工作目錄尋找指令檔。
- 合併：由外到內合併；越接近目前工作目錄的規則越後讀，因此能覆蓋較上層的規則。
- 大型專案：可在不同 package / service 放自己的 AGENTS.md，避免把所有規則都塞在根目錄。

## 4. 第一份 AGENTS.md：建議模板

這是什麼專案？用什麼技術？怎麼測試？哪些事情不能做？什麼才叫完成？

```
# AGENTS.md

## Project overview
- 說明這個專案的目的與主要技術。

## Development rules
- 指定語言版本、主要套件與程式風格。
- 優先修改既有程式，不要無故重寫整個專案。

## Testing
- 修改完成後執行指定測試。
- 若測試失敗，先判斷是否由本次修改造成並修正。

## Boundaries
- 不要修改原始資料檔。
- 不要把 API key、密碼或 token 寫進程式碼。
- 部署、刪除大量檔案等高影響操作先詢問。

## Definition of done
- 程式可執行。
- 相關測試通過。
- 簡要說明修改了哪些檔案與原因。
```

### 寫法原則：短、具體、可驗證

| 較差的寫法 | 較好的寫法 |
| --- | --- |
| Write good code. | Python 使用 3.12；函式名稱使用 snake_case；新增公開函式要有 docstring。 |
| Test everything. | 修改 parser.py 後執行 pytest tests/test_parser.py。 |
| Do not break the project. | 不要改 input_data/；完成後執行 pytest 與 python main.py --demo。 |
| Read all docs before every edit. | 修改資料庫 schema 時讀 docs/database.md；準備部署時讀 docs/deploy.md。 |

近年的 Agent 能自行判斷許多一般性步驟，

AGENTS.md 應避免堆疊過多「每次都要做」的儀式化流程。

把規則留給重要、具專案特性的限制，通常更有效率。

## 5. 範例情境一：Python 資料分析作業

情境：學生拿到一份 CSV，要用 Python 做清理、統計與視覺化。

老師希望 AI 可以協作，但不能修改原始資料，也不能額外安裝套件。

**AGENTS.md 範例**

```
# AGENTS.md

## Project
This is a Python data-analysis assignment.

## Rules
- Use Python 3.12.
- Prefer pandas and matplotlib.
- Do not modify files under data/raw/.
- Save cleaned data to data/processed/.
- Save figures to outputs/figures/.
- Do not add new dependencies unless they are necessary.

## Validation
- Run `python main.py` before finishing.
- Confirm the program can run from a clean checkout.
- Briefly explain the analysis choices in README.md.
```

**這份規則解決什麼？** Agent 知道資料不可覆寫、產出位置固定，而且完成前要真的執行程式驗證。

## 6. 範例情境二：前端 + Firebase 小專題

情境：學生做一個 GitHub Pages 網頁，Firebase 負責資料儲存。希望 AI 協助開發，但不能把 Firebase 金鑰或管理憑證直接寫入版本庫，也不要任意換框架。

**AGENTS.md 範例**

```
# AGENTS.md

## Architecture
- Frontend-only web application.
- Use the existing HTML/CSS/JavaScript structure.
- Firebase is used for application data.

## Development rules
- Preserve the current UI structure unless the task requests a redesign.
- Reuse existing modules before creating new abstractions.
- Never commit service-account credentials, private keys, or secrets.
- Do not migrate the project to another framework unless explicitly requested.

## Before finishing
- Check the browser console for errors.
- Verify create/read/update flows against the development Firebase project.
- List changed files and any configuration the user must provide manually.
```

## 7. 範例情境三：檔案式資料庫課程任務

情境：課程目的就是讓學生理解「資料庫系統需要哪些能力」，因此刻意要求不能用 SQL / SQLite，而是自己用檔案實作查詢、修改與整理。這類限制非常適合寫進 AGENTS.md，避免 Agent 一看到資料庫題目就直接導入現成資料庫。

**AGENTS.md 範例**

```
# AGENTS.md

## Learning goal
This assignment demonstrates database concepts by building a file-based system.

## Hard constraints
- Do NOT use SQL, SQLite, PostgreSQL, MySQL, ORM, or embedded databases.
- Store records in CSV or JSON files.
- Implement data operations in Python.

## Required capabilities
- Insert a record.
- Query records by at least two fields.
- Update an existing record.
- Delete a record.
- Sort or organize records.
- Handle invalid input without corrupting the data file.

## Engineering rules
- Keep data access functions separate from the CLI/UI layer.
- Do not rewrite the entire file when a smaller safe change is sufficient, unless the design requires it.
- Include a small demo dataset and repeatable test commands.

## Definition of done
- `python app.py --demo` completes without errors.
- Demonstrate create, query, update, and delete operations.
- Explain which parts correspond to common database-system concepts.
```

## 8. 範例情境四：團隊專題，限制 Agent 的變更範圍

**AGENTS.md 範例**

```
# AGENTS.md

## Scope
- Make the smallest change that satisfies the task.
- Do not rename public APIs unless the task explicitly requires it.
- Do not edit generated files under dist/ or build/.

## Team workflow
- Keep commits focused on one logical change.
- Add or update tests for changed behavior.
- Do not deploy to production automatically.
- For schema migrations or destructive changes, stop and explain the risk before proceeding.

## Completion
- Run affected tests.
- Summarize changed files, test results, and remaining risks.
```

## 9.根目錄規則 + 子目錄規則

當專案有 frontend、backend、mobile 等不同子專案時，可讓根目錄放共同規則，再由子目錄補充自己的規則。

**目錄結構**

```
project/
├─ AGENTS.md
├─ frontend/
│  ├─ AGENTS.md
│  └─ src/
└─ backend/
   ├─ AGENTS.md
   └─ app/
```

**根目錄：全專案共同規則**

```
# project/AGENTS.md
## Common rules
- Never commit secrets.
- Keep changes focused.
- Summarize tests before finishing.
```

**frontend：局部規則**

```
# project/frontend/AGENTS.md
## Frontend rules
- Use TypeScript strict mode.
- Preserve accessibility labels.
- Run `npm run lint` and `npm test` after UI logic changes.
```

**規則衝突時：** 在 Codex 的指令鏈中，越接近目前工作目錄的專案規則會出現在較後面，因此可用來覆蓋較上層的指令。

## 10. AGENTS.md 不適合放什麼？

- 大量與目前專案無關的背景知識：會占用上下文，反而降低重點。
- 會頻繁變動的單次任務：應放在這次 Prompt，而不是長期規則。
- 密碼、API Key、token、私密連線資訊：不要把 secret 寫入 AGENTS.md。
- 過度模糊的口號：例如「寫出最好的程式」、「永遠保持高品質」。
- 彼此矛盾的規則：例如一處要求永遠不要新增套件，另一處又要求缺套件就自行安裝。